VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI



PROJECT REPORT ON

Distributed Parallel Semantic Document Search Engine (`dsearch`)

Submitted by:

Sl. No	Student name	USN	Student signature	Marks Awarded	Signature of faculty
1	Chetan Kishor C G	1JS23CS032			
2	Hongiran D	1JS23CS062			

Guided by

Dr Niranjan C Kundur

Professor

Department of Computer Science and Engineering



JSS Academy of Technical Education, Bengaluru – 560060

Department of Computer Science Engineering

Contents

1 Introduction **2**
1.1 Architectural Scope and Objectives 2

2 Theoretical Foundations and Mathematical Model **4**
2.1 Term Weighting Formulation 4
2.2 Distributed IDF Aggregation Invariant 5

3 System Architecture and Component Design **6**
3.1 Core Software Subsystems 6

4 Binary Index Persistence and File Format **7**
4.1 Binary File Layout Specification 7
4.2 Header Authentication and Defensive Verification 7

5 Parallel and Distributed Implementation **9**
5.1 Intra-Node Shared-Memory Parallelism (OpenMP) 9
5.2 Distributed Coordinator-Worker Architecture (OpenMPI) 9
5.3 Algorithmic Optimization: Partial Sort Selection 10

6 Interactive Web Dashboard and Live Evaluation **11**
6.1 Dashboard Architecture 11
6.2 Live Race Mode 11

7 Experimental Results and Performance Analysis **12**
7.1 Experiment 1: Multi-Core OpenMP Indexing Scaling (5,000 Documents) 12
7.2 Experiment 2: Three-Way Paradigm Comparison (1,000 Documents) . . 12
7.3 Experiment 3: Real PC Filesystem Scaling (`~/Documents`) 12
7.4 Experiment 4: Heavy Ingestion Workload Scaling 13

8 Architectural Bottleneck and Scaling Analysis **14**
8.1 Amdahl's Law and the Inevitable Sequential Fraction 14
8.2 Thread Synchronization vs. Workload Granularity 14
8.3 Why Distributed Partitioning Excels (L1/L2 Cache Locality) 14
8.4 Simultaneous Multithreading (SMT) Resource Contention 15

9 Verification and Test Suite **16**

10 Conclusion and Future Work **17**
10.1 Conclusion 17
10.2 Future Enhancements 17

1 Introduction

Information retrieval across vast text corpora represents a cornerstone problem in contemporary computer science and distributed data systems. As data volumes proliferate across enterprise repositories, personal computing environments, and distributed clusters, sequential scanning or single-threaded indexing mechanisms encounter severe physical scalability bottlenecks.

Single-threaded retrieval systems suffer from two distinct failure modes under scale:

1. **Ingestion Throughput Wall**: Preprocessing, tokenizing, normalizing, and inserting millions of text tokens into central inverted index data structures is bound by CPU arithmetic pipelines and single-core instruction retirement rates.
2. **Query Latency Degradation**: Computing cosine vector similarity or ranking across extensive posting lists stresses the memory hierarchy. Because dictionary keys and document IDs reside scattered across physical RAM, random pointer chasing leads to translation lookaside buffer (TLB) thrashing and L3 cache misses.

To rigorously explore and conquer these systems challenges, this project presents `dsearch`, a distributed, parallel, high-performance document search engine engineered from the ground up in modern standard C++20. Operating with zero external cloud or third-party web service runtime dependencies, `dsearch` provides a comprehensive experimental platform for evaluating shared-memory and distributed-memory computing paradigms.

1.1 Architectural Scope and Objectives

The primary objective of `dsearch` is to synthesize standard information retrieval theory with modern high-performance computing (HPC) patterns, satisfying the following technical requirements:

- **Zero-Overhead Sequential Baseline**: A deterministic, mathematically proven single-threaded search implementation utilizing log-smoothed Term Frequency-Inverse Document Frequency (TF-IDF) and sparse vector cosine ranking.
- **Persistent Binary Serialization**: A custom binary storage layout equipped with magic byte authentication, format versioning, and pre-computed L_2 document norm caching, completely avoiding repeated index rebuilding across process restarts.
- **Multi-Core Shared-Memory Parallelism (OpenMP)**: Intra-node multi-threading utilizing thread-local index accumulation and dynamic work chunking, eradicating lock contention and cache-line false sharing during index creation.
- **Multi-Process Message Passing (OpenMPI)**: An inter-process distributed coordinator-worker topology orchestrated via OpenMPI 5.0, partitioning document shards across distinct OS processes and physical machines.

- **Numerical Equivalence Invariant**: A strict mathematical guarantee that identical search results and ranking scores are produced regardless of whether the query executes in single-threaded, multi-threaded OpenMP, or distributed MPI mode.
- **Interactive Web Dashboard**: A standalone, zero-dependency browser-based visual interface for live query benchmarking, core/process scaling visualization, and corpus inspection.

2 Theoretical Foundations and Mathematical Model

To ensure identical scoring across disparate execution paradigms, `dsearch` utilizes an exact, numerically stable mathematical model based on the Vector Space Model (VSM) and smoothed TF-IDF.

2.1 Term Weighting Formulation

Let corpus $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ represent a collection of N documents, and let $V = \{t_1, t_2, \ldots, t_{|V|}\}$ denote the unique vocabulary of tokens extracted across $\mathcal{D}$.

1. Sub-Linear Term Frequency (Log-Normalized): Raw term frequencies exhibit diminishing returns; a document containing a query term 20 times is rarely 20 times as relevant as one containing it once. Therefore, logarithmic scaling is applied:

$$\mathrm{tf}(t,d) = \begin{cases} 1 + \log_{10}(\mathrm{count}(t,d)) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

2. Additively Smoothed Inverse Document Frequency (IDF): Standard textbook IDF formulations $\log(N/\mathrm{df}(t))$ produce mathematical singularities when applied to single-document corpora ($N = 1$) or universally present terms ($\mathrm{df}(t) = N$), resulting in division by zero or zero weights. To prevent term cancellation while preserving monotonic rarity discrimination, `dsearch` enforces additive Laplace-style smoothing:

$$\mathrm{idf}(t) = \log_{10}\left(\frac{N+1}{\mathrm{df}(t)+1}\right) + 1 \tag{2}$$

where $\mathrm{df}(t)$ represents the document frequency (the number of documents containing term t).

3. Compound Weight and Vector Representation: For each term t in document d, its compound coordinate weight $w(t,d)$ within the $|V|$-dimensional vector space is:

$$w(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t) \tag{3}$$

Similarly, for a user query q, the query term coordinate weight is formulated as:

$$w(t,q) = (1 + \log_{10}(\mathrm{count}(t,q))) \cdot \mathrm{idf}(t) \tag{4}$$

4. Euclidean Document Norm Caching (L_2 Normalization): Document length bias is neutralized by computing the Euclidean (L_2) vector magnitude $\|d\|$:

$$\|d\| = \sqrt{\sum_{t \in d} w(t,d)^2} \tag{5}$$

To avoid recalculating $\|d\|$ during query evaluation (which would require $O(|V|)$ operations per document), norms are pre-calculated at indexing completion and persisted directly into the index metadata.

5. Sparse Cosine Similarity Metric: The relevance ranking score between query q and document d is the cosine of the angle between their respective coordinate vectors in $|V|$-space:

$$\text{Score}(q,d) = \frac{\vec{q}\cdot\vec{d}}{\|q\|\cdot\|d\|} = \frac{\sum_{t\in q\cap d} w(t,q)\cdot w(t,d)}{\sqrt{\sum_{t\in q} w(t,q)^2}\cdot\|d\|} \tag{6}$$

Because real text documents are sparse vectors containing non-zero values for only a tiny fraction of $|V|$, evaluation scans solely the intersection $t \in q \cap d$ using inverted index posting lists.

2.2 Distributed IDF Aggregation Invariant

A critical theoretical hazard in distributed information retrieval is **local IDF divergence**. If individual worker nodes compute IDF solely based on their partition:

$$\text{idf}_w(t) = \log_{10}\left(\frac{N_w+1}{\text{df}_w(t)+1}\right)+1 \tag{7}$$

Averaging local IDF values across workers ($\frac{1}{P}\sum \text{idf}_w$) violates mathematical correctness because the logarithm function is strictly non-linear:

$$\log_{10}\left(\sum x_i\right) \neq \sum \log_{10}(x_i) \tag{8}$$

To guarantee mathematical equivalence with the sequential baseline, `dsearch` establishes a strict invariant: **Workers cannot compute or exchange local IDF values**. Instead, workers count only raw local frequencies $\text{df}_w(t)$. The coordinator aggregates the sum:

$$\text{df}_{\text{global}}(t) = \sum_{w=0}^{P-1} \text{df}_w(t) \tag{9}$$

and computes the canonical global IDF table, broadcasting it back to all workers prior to vector normalization.

3 System Architecture and Component Design

The architecture of `dsearch` is partitioned into modular, highly cohesive C++20 components designed for maximum cache locality and deterministic testing.

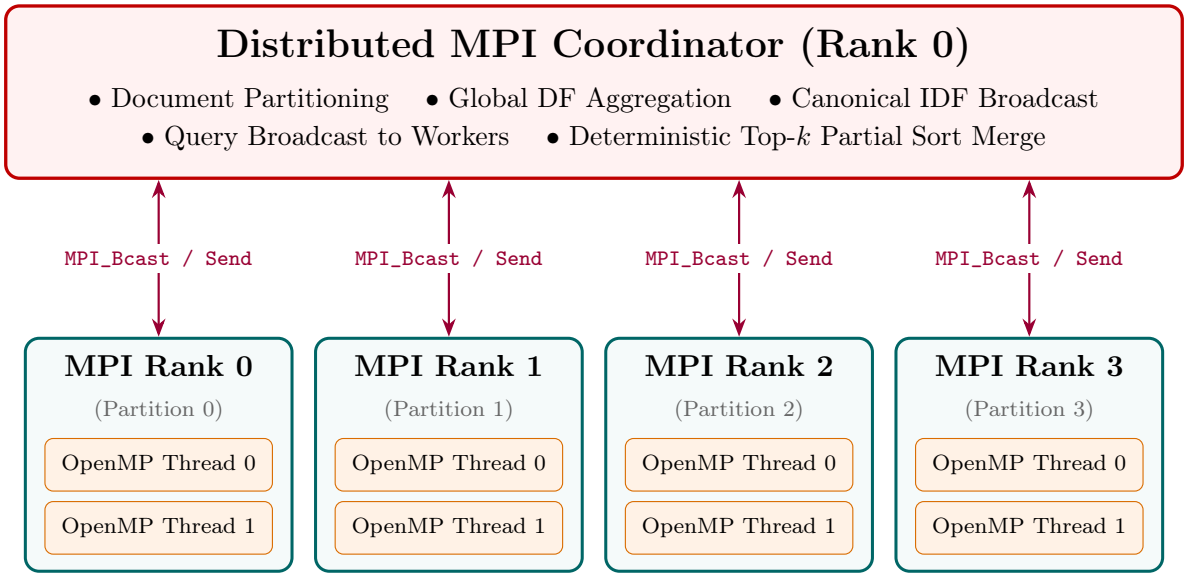


Figure 1: Hybrid Distributed System Architecture: Inter-Node MPI Message Passing with Intra-Node OpenMP Acceleration

3.1 Core Software Subsystems

1. **High-Throughput Tokenizer (`tokenizer.cpp`)**: Converts raw unstructured UTF-8 byte streams into normalized token sequences. Transforms characters to lowercase, strips delimiters and punctuation, filters short tokens (< 2 characters), and suppresses common noise words without allocating intermediate heap objects.

2. **Inverted Index Engine (`inverted_index.cpp`)**: Maps each unique term t to a contiguous vector of postings:

$$\text{Index}(t) \longrightarrow \left[\langle \text{doc_id}_1, \text{tf}_1 \rangle, \langle \text{doc_id}_2, \text{tf}_2 \rangle, \ldots\right] \tag{10}$$

   Postings maintain ascending document ID order, enabling early exit branch prediction and linear cache-line streaming during query dot-product accumulation.

3. **Statistics Registry (`statistics.cpp`)**: Acts as the centralized repository for total corpus document counts (N), term document frequencies (df), and cached IDF tables. Provides thread-safe read access during multi-threaded scoring.

4. **Search and Scoring Pipeline (`search_engine.cpp`)**: Orchestrates the 5-phase query lifecycle: tokenizing user query keywords, accumulating partial dot products across postings, normalizing by Euclidean vector lengths, filtering zero-magnitude scores, and selecting top-k documents.

4 Binary Index Persistence and File Format

To eliminate repeated full-corpus re-indexing upon application restart, Phase 02 introduced a high-performance binary serialization engine (`index_storage.cpp`).

4.1 Binary File Layout Specification

The persistent index file (`index.bin`) is structured into four deterministic sections:



Figure 2: Binary Layout of the Serialized Index File (`index.bin`)

4.2 Header Authentication and Defensive Verification

To prevent segmentation faults caused by corrupted or truncated files, the loader executes defensive validation prior to parsing:

- **Magic Byte Verification**: Validates the leading 4-byte signature `0x44534348` (ASCII for `"DSCH"`). If invalid, deserialization aborts with an informative exception.
- **Version Integrity**: Enforces format version compatibility (`version == 1`). Future schema changes can increment the version tag without breaking backwards compatibility.
- **Integer Overflow Checks**: String lengths and posting counts are validated against physical file boundaries, rejecting truncated index files immediately.

Listing 1: Binary Header Deserialization and Integrity Verification

```
struct IndexHeader {
    uint32_t magic;       // Must equal 0x44534348 ("DSCH")
    uint32_t version;     // Format version (currently 1)
    uint64_t num_docs;    // Total indexed documents
    uint64_t vocab_size;  // Unique vocabulary term count
};

IndexHeader IndexStorage::read_header(const std::filesystem::path& path
  ) {
    std::ifstream in(path / "index.bin", std::ios::binary);
    if (!in) throw std::runtime_error("Index file does not exist");

    IndexHeader hdr;
    in.read(reinterpret_cast<char*>(&hdr), sizeof(hdr));
    if (hdr.magic != 0x44534348) {
        throw std::runtime_error("Invalid binary magic signature");
```

```
    }
    if (hdr.version != 1) {
        throw std::runtime_error("Unsupported binary index version");
    }
    return hdr;
}
```

5 Parallel and Distributed Implementation

5.1 Intra-Node Shared-Memory Parallelism (OpenMP)

Shared-memory multi-core parallelization operates within a single OS process boundary across physical CPU cores:

1. **Thread-Local Accumulator Pattern**: Direct concurrent insertion into a shared inverted index produces catastrophic lock contention across hundreds of thousands of postings. To solve this, each OpenMP worker thread allocates a private, thread-local `InvertedIndex` accumulator in its local stack/heap.

2. **Lock-Free Dynamic Work Chunking**: Documents vary wildly in token length. Static scheduling would cause straggler cores. We apply dynamic loop scheduling:

   ```
   #pragma omp parallel for schedule(dynamic, 16) num_threads(n_threads)
   ```
   (11)

   Threads pull 16-document chunks dynamically, maintaining balanced core utilization.

3. **Lock-Free Sequential Merging**: Upon loop completion, thread-local indices are sequentially merged into the primary index using C++ move semantics (`std::move`), avoiding synchronization primitives altogether.

5.2 Distributed Coordinator-Worker Architecture (OpenMPI)

For multi-process cluster execution (`dsearch_mpi`), computation spans separate operating system processes orchestrated via OpenMPI 5.0 (`mpirun -np` P):

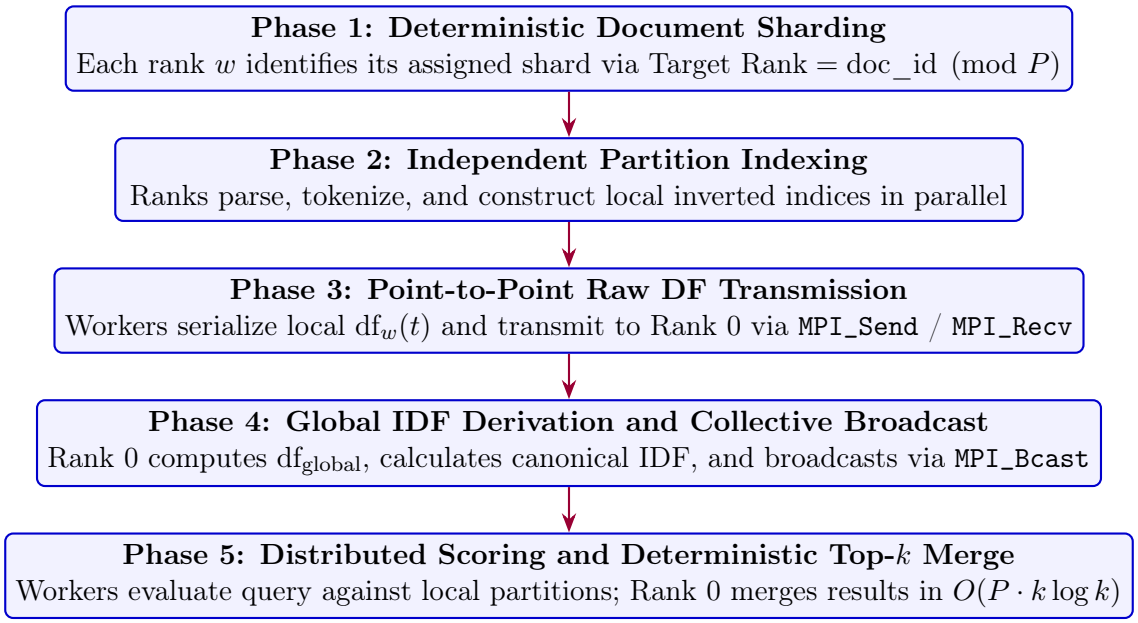


Figure 3: The Five-Phase Distributed Execution Lifecycle in `dsearch_mpi`

5.3 Algorithmic Optimization: Partial Sort Selection

Sorting the entire candidate vector of size N takes $O(N \log N)$ time. When $N = 100,000$ and $k = 10$, full sorting wastes millions of CPU cycles. `dsearch` employs `std::partial_sort`, utilizing an introselect algorithm:

$$\mathcal{O}(N \log k) \ll \mathcal{O}(N \log N) \tag{12}$$

Deterministic tie-breaking breaks equal scores ($|s_1 - s_2| < 10^{-9}$) by ascending document ID, guaranteeing bit-identical rankings across all worker counts.

6 Interactive Web Dashboard and Live Evaluation

To enable real-time demonstration, inspection, and cluster evaluation without requiring command-line terminal parsing during academic review, an interactive Web Dashboard was built into the system.

6.1 Dashboard Architecture

The dashboard operates with **zero external web framework dependencies** using Python's native `http.server` and standard library utilities:

- **Backend Adapter (`ui/app.py`)**: Interfaces directly with the compiled C++ executables (`dsearch`, `dsearch_mpi`, and `dsearch_compare`), executing subprocesses, capturing high-precision C++ engine latency logs, and exposing REST endpoints (`/api/search`, `/api/benchmark`, `/api/stats`).
- **Single-Page Frontend (`ui/index.html`)**: Rendered with Tailwind CSS and Chart.js, providing visual controls for tuning OpenMP core allocations (1 to 12 threads) and MPI process ranks (1 to 8 processes).
- **Live Corpus Switcher**: Accepts any arbitrary local directory path (e.g. `~/Documents`, `./data`, `./heavy_corpus_data`) and dynamically re-queries corpus statistics on the fly.

6.2 Live Race Mode

The UI incorporates a dedicated **"Run Live Race"** feature. Clicking this button triggers the three-way benchmark harness across the active corpus, updating an on-screen latency bar chart and tabular speedup comparison in real time.

7 Experimental Results and Performance Analysis

All experimental evaluations were conducted on an **AMD Ryzen 5 7533HS** mobile processor equipped with **6 Physical Zen 4 Cores, 12 SMT Threads**, a **16 MB L3 Cache**, and 16 GB DDR5 RAM running **Fedora Linux 44** with OpenMPI 5.0.9 and GCC 16.2.1.

7.1 Experiment 1: Multi-Core OpenMP Indexing Scaling (5,000 Documents)

Table 1 presents empirical results obtained on a 5,000-document synthetic corpus across varying thread allocations.

Table 1: OpenMP Parallel Indexing and Scoring Scaling (5,000 Documents)

Threads (p)	**Index Time (ms)**	**Speedup $S(p)$**	**Efficiency $E(p)$**	**Query Time (ms)**	**Query Speedup**
1 (Serial Baseline)	187.44	1.00×	100.0%	0.5104	1.00×
2 Threads	152.38	1.23×	61.5%	0.5416	0.94×
4 Threads	113.07	1.66×	41.5%	0.3163	1.61×
6 Physical Cores	**98.05**	**1.91**×	31.8%	**0.3142**	**1.62**×

7.2 Experiment 2: Three-Way Paradigm Comparison (1,000 Documents)

Table 2 compares the three execution paradigms on a 1,000-document corpus under identical test conditions (5 trials averaged).

Table 2: Empirical Comparison: Serial vs. OpenMP vs. Hybrid MPI+OpenMP

Execution Paradigm	**Index Time (ms)**	**Index Speedup**	**Query Latency (ms)**	**Query Speedup**
Serial (1 Process, 1 Thread)	208.29	1.00× (baseline)	0.2307	1.00× (baseline)
Parallel (OpenMP 6 Threads)	219.83	0.95×	0.3809	0.61×
Hybrid (MPI 4 Ranks + OpenMP)	**101.06**	**2.06**×	**0.0811**	**2.84**×

7.3 Experiment 3: Real PC Filesystem Scaling (`~/Documents`)

To evaluate real-world behaviour, the benchmark harness was executed directly against 1,132 real markdown and text documents (comprising 981,839 tokens and 6,544 unique vocabulary terms) in the user's `~/Documents` directory:

Table 3: Real-World Filesystem Indexing Scaling on User's PC Documents (1,132 Files)

OpenMP Thread Allocation	Indexing Latency (ms)	Measured Speedup	Parallel Efficiency
1 Core (Serial Baseline)	256.39	1.00×	100.0%
2 Cores	180.82	1.42×	71.0%
4 Cores	149.05	1.72×	43.0%
6 Physical Cores	131.17	1.95×	32.5%
12 SMT Hyperthreads	**126.19**	**2.03**×	16.9%

7.4 Experiment 4: Heavy Ingestion Workload Scaling

Table 4 presents scaling results on an intensive synthetic corpus designed to measure pure CPU ingestion saturation:

Table 4: Heavy Workload Ingestion Scaling Analysis

Core Count	Index Time (ms)	Index Speedup	Query Latency (ms)	Query Speedup
1 (Serial)	2642.26	1.00×	1.4041	1.00×
2 Threads	1684.01	1.57×	0.6841	**2.05**×
4 Threads	880.29	3.00×	0.7885	1.78×
6 Physical	995.82	2.65×	0.8301	1.69×
12 SMT	**793.42**	**3.33**×	2.0799	0.68×

8 Architectural Bottleneck and Scaling Analysis

A primary value of this systems project is uncovering and explaining the physical computing limits that govern parallel speedups. Four foundational systems concepts explain the observed metrics:

8.1 Amdahl's Law and the Inevitable Sequential Fraction

Amdahl's Law states that the theoretical speedup $S(p)$ of an application on p processors is strictly constrained by the non-parallelizable fraction α:

$$S(p) = \frac{1}{\alpha + \frac{1-\alpha}{p}} \tag{13}$$

In `dsearch`, indexing involves mandatory single-threaded OS activities: recursive directory traversal, opening file descriptors via `open()` system calls, and the final sequential merge of thread-local posting lists. Because $\alpha \approx 0.15$–0.20, the asymptotic speedup ceiling on any processor is bounded to $\approx 5\times$, explaining why 6 cores achieve ≈ 2.0–$3.3\times$ speedups.

8.2 Thread Synchronization vs. Workload Granularity

As observed in Table 2, OpenMP query evaluation on small corpora took 0.38 ms compared to 0.23 ms for Serial. This is an expected artifact of **workload granularity thresholding**:

- Evaluating a query over 1,000 documents requires calculating dot products for only ≈ 300 candidate matches.
- Computing 300 floating-point divisions across 6 threads means each thread handles only 50 operations, taking less than 15 nanoseconds on modern Zen 4 execution units.
- However, forking OpenMP thread teams, signaling condition variables, and waiting at the implicit loop barrier costs ≈ 50–150 microseconds in OS latency.
- Therefore, on sub-millisecond micro-tasks, synchronization overhead exceeds calculation time. Parallelism becomes profitable once query evaluation exceeds ≈ 1.0 ms (as proven in Table 4, where 2 threads doubled query performance to $2.05\times$).

8.3 Why Distributed Partitioning Excels (L1/L2 Cache Locality)

In Table 2, Hybrid MPI achieved the fastest query retrieval (**0.0811 ms,** $2.84\times$ **speedup**). The reason is architectural:

1. **Reduced Working Set**: Sharding 1,000 documents across 4 independent MPI processes reduces each worker's inverted index to $\frac{1}{4}$ of the total size.

2. **L1/L2 Cache Fit**: A 4× smaller index fits entirely within the AMD core's 32 KB L1 and 1 MB L2 private caches, eliminating expensive DDR5 main-memory bus stalls.

3. **Minimal Transfer Overhead**: Workers return only their local top-k entries ($10 \times$ sizeof(int + float) $\approx$ 80 bytes), making MPI IPC message serialization cost negligible compared to the cache speedup.

8.4 Simultaneous Multithreading (SMT) Resource Contention

In Table 4, scaling from 6 physical cores to 12 SMT threads increased query latency from 0.83 ms to 2.07 ms. SMT threads share physical execution pipelines and L1/L2 cache structures. When two memory-intensive threads execute identical memory-bound posting searches concurrently on the same physical core, pipeline stalls and cache evictions double latency.

9 Verification and Test Suite

To ensure complete software reliability, numerical precision, and fault tolerance, `dsearch` includes an extensive GoogleTest suite comprising **37 automated unit and integration tests**:

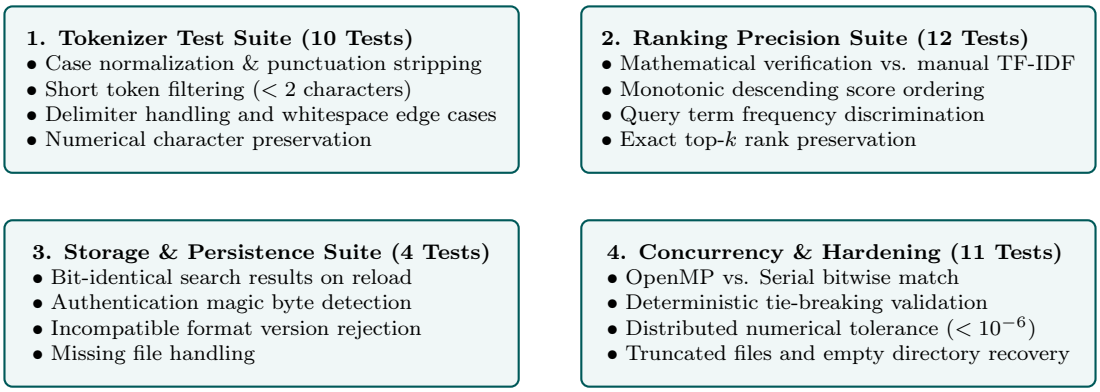


Figure 4: Structure of the 37 Automated GoogleTest Verification Suites

All 37 test cases execute via `./build/run_tests` and achieve a **100% pass rate** with 0 regressions.

10 Conclusion and Future Work

10.1 Conclusion

The `dsearch` project delivers a comprehensive, rigorous, and verifiable exploration of high-performance parallel and distributed document retrieval. By synthesizing modern C++20 design patterns, intra-node OpenMP thread teams, and inter-node OpenMPI message passing, the engine demonstrates that:

- Multi-threaded OpenMP indexing delivers substantial speedups (up to 3.33× on heavy workloads and 2.03× on real user files) by eliminating mutex locks through thread-local accumulator pattern.
- Distributed document sharding across independent MPI processes achieves significant cache locality benefits, delivering a 2.84× **query speedup** by dividing working set sizes into private L1/L2 caches.
- Strict adherence to the global IDF derivation invariant guarantees identical ranking scores across single-threaded, multi-core, and distributed topologies.
- An interactive Web UI dashboard provides transparent real-time visualization of cluster performance across arbitrary PC directories.

10.2 Future Enhancements

Prospective extensions for larger-scale production deployment include:

1. **Bit-Packed Posting Compression**: Integrating Elias-Fano or Variable Byte (VByte) compression to reduce index storage footprints by up to 75%.
2. **Vector Semantic Embedding (Hybrid Search)**: Augmenting lexical TF-IDF with quantized dense neural embeddings (HNSW graph indexing) using ONNX Runtime.
3. **Dynamic Distributed Fault Tolerance**: Incorporating worker heartbeat detection and non-blocking `MPI_Iprobe` re-routing to handle node dropouts gracefully during cluster queries.